

Advanced Web Programming

SIB245007 | D-IV Sistem Informasi Bisnis

Meeting 5: **Eloquent ORM and Relationship**

Mapping Objects to Tables, Relationships Between Models, and Efficient Queries

What You'll Learn

An Eloquent model can do far more than just retrieve and save data one row at a time. This meeting covers:

1. What **ORM** is, and how Eloquent guesses table names and runs basic operations (fetch, create, update, delete) without manual SQL
2. How to connect models through **relationship** (`hasOne` , `hasMany` , `belongsTo` , `belongsToMany`)
3. Why a carelessly loaded relationship can silently fire off dozens of queries (**N+1**), and how to avoid it with **eager loading**
4. How to display thousands of rows without overloading a single page, through **pagination**

This slide deck covers concepts. Applying all of this to Simple POS happens in the practicum jobsheet.

Part 1

Eloquent Fundamentals

The ORM Concept: Mapping Classes to Tables

Why does `Article::find(1)` immediately know to fetch from the `articles` table, when you never wrote that table name anywhere? The answer is ORM.

ORM (Object-Relational Mapping): a technique that maps a class in code to a table in the database, and an object to a row, so database operations are written through methods and object properties instead of raw SQL queries.



Eloquent uses the **Active Record** pattern: one model class represents one table, while also providing the methods to query, save, and delete its own data, not a separate class for every task.

The Eloquent Model: One Class per Table

```
class Article extends Model
{
    //
}
```

It looks empty, but don't be fooled: this class can already fetch, save, update, and delete data without a single line of SQL written anywhere. All of that comes free from one line, `extends Illuminate\Database\Eloquent\Model`.

- One model class represents one table; one object from a query represents one row in that table
- A model this "empty" is normal, most of its behavior is already automatic through convention (next slide)

Eloquent Conventions: An Automatic Guess, Not a Coincidence

The plural snake_case table names you've been writing in migrations all along aren't a coincidence. Eloquent guesses three things from the model class's name alone, with zero configuration from you.

Model	Table (convention)	Primary Key	Timestamps
Article	articles	id (automatic)	created_at / updated_at (automatic)
Category	categories	id (automatic)	created_at / updated_at (automatic)

The process: take the class name, convert it to snake_case, then pluralize it. Category becomes categories. Later in the jobsheet you'll create a TransactionDetail model, and by that same rule Eloquent will look for it in a transaction_details table, without you saying a word.

Retrieving Data: Basic Eloquent Queries

```
$articles = Article::all();  
$article = Article::find(1);  
$articles = Article::where('title', 'like', '%laravel%')->get();  
$article = Article::where('title', 'like', '%laravel%')->first();
```

Four lines, two different shapes of result. `all()` and `get()` return a **Collection**, which can hold many model objects at once. `find()` and `first()` return just **one** model object, or `null` if nothing matched, so always plan for that `null` case.

Creating Data: `create()` vs `new` + `save()`

There are two ways to write a new row, and both end up exactly the same in the database.

Via `create()`

```
Article::create([
  'title' => 'Judul Baru',
  'body' => 'Isi artikel...',
]);
```

Via `new` + `save()`

```
$article = new Article();
$article->title = 'Judul Baru';
$article->body = 'Isi artikel...';
$article->save();
```

`create()` is shorter and more commonly used, but there's a trap that shows up the moment you try it: Eloquent can reject it outright.

Mass Assignment and `$fillable`

Try `Article::create([...])` on a fresh model straight out of `make:model`, and Eloquent throws `MassAssignmentException`. This isn't a bug.

Mass Assignment: filling in many model columns at once through a single array, like in `create()`. Eloquent blocks it by default via `MassAssignmentException`, unless which columns are allowed to be mass-filled has been registered first.

```
class Article extends Model
{
    protected $fillable = ['title', 'body'];
}
```

`$fillable` is an allowlist of columns that can be mass-filled. Without it, Eloquent rejects mass `create()` / `update()` by default, so sensitive columns like a role or a status column can't quietly get filled through unexpected input.

Updating and Deleting Data

```
$article = Article::find(1);  
$article->title = 'Judul Diperbarui';  
$article->save();  
  
// or directly in one call:  
Article::find(1)->update(['title' => 'Judul Diperbarui']);  
  
Article::find(1)->delete();
```

Same pattern as creating data: change a property then `save()`, or go straight through one `update()` call. `delete()` has no "undo," so get in the habit of `find()`-ing and checking first before calling it in real code.

Full method list: [official Laravel Eloquent documentation](#) (`firstOrCreate` , `updateOrCreate` , and more)

Part 2

Relationships Between Models

Without Relationship vs. With Eloquent Relationship

Without an Eloquent relationship

- Every time related data is needed, hand-write the join query again
- Prone to typos in join column names, repeated in many places

With an Eloquent relationship

- Declared once, in the model
- Used anywhere through a property or method, no rewriting the join

Similar to a family tree: the relationships get drawn once, and anyone looking at it instantly understands how everyone connects, no digging through separate records.

Relationship: Connecting Models

Relationship: a method on an Eloquent model describing how one table connects to another, used like an ordinary property even though it's really running a query behind the scenes.

```
class Author extends Model
{
    public function articles(): HasMany
    {
        return $this->hasMany(Article::class);
    }
}

class Article extends Model
{
    public function author(): BelongsTo
    {
        return $this->belongsTo(Author::class);
    }
}
```

hasOne: A One-to-One Relationship

Swap `hasMany` for `hasOne` , and Eloquent immediately knows to stop at the first row only:

```
class Author extends Model
{
    public function profile(): HasOne
    {
        return $this->hasOne(Profile::class);
    }
}
```

One author has one profile, not several, so `$author->profile` returns a single object directly, not a Collection.

belongsToMany: A Many-to-Many Relationship

```
class Article extends Model
{
    public function tags(): BelongsToMany
    {
        return $this->belongsToMany(Tag::class);
    }
}
```

One article can have many tags, and one tag can be used on many articles, so no single foreign key column is enough. It needs a third table in between, `article_tag`, following the naming convention: both model names singular, alphabetical order, joined by an underscore.

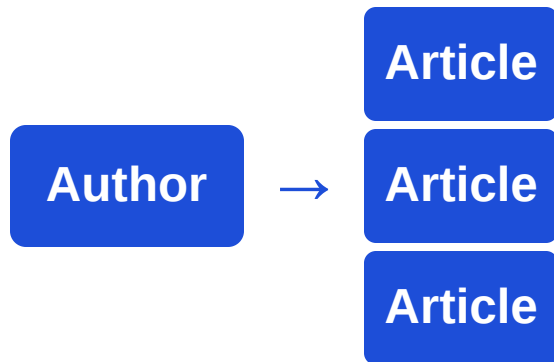
Relationship Shapes: Four Basic Patterns

The four methods you just saw are really only two ideas: who has one, and who has many, plus the reverse direction.

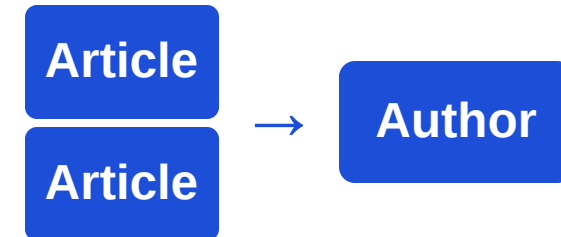
hasOne: one parent, one child



hasMany: one parent, many children



belongsTo: the reverse of hasMany



belongsToMany: many meets many



Four Relationship Types: A Summary

Relationship	Meaning	Example
hasOne	One parent row has one related row	Author has one Profile
hasMany	One parent row has many related rows	Author has many Articles
belongsTo	This row points back to one parent row	Article points to one Author
belongsToMany	Many-to-many through a pivot table	Article and Tag connect both ways

Why a Relationship's Name Matters

Write `articles()` once on the `Author` model, and that name immediately applies in three different places:

- **Method call** `$author->articles()` : returns a query builder, chainable with `->where()` , `->orderBy()` , etc.
- **Property access** `$author->articles` : returns the loaded collection directly, no parentheses
- **String inside** `with('articles')` : tells Eloquent which relationship to load up front, the next slide's topic

Part 3

Eager Loading and the N+1 Problem

The N+1 Problem

A relationship that's this easy to use can quietly become the most common performance problem in any Laravel app, and its name is oddly specific: N+1.

Lazy Loading: a relationship only loads once that property is actually accessed, row by row, the root cause of the N+1 problem.

N+1: a problematic query pattern, one query for the main listing, plus one separate query for every single row to load its relationship.

Displaying 15 articles with each one's author name, without eager loading: 1 list query + 15 author queries = **16 queries**, just for one page of 15 rows.

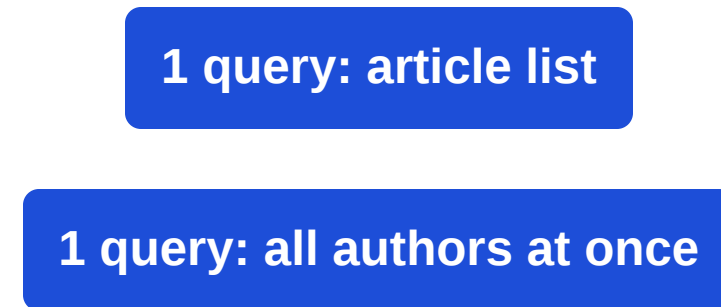
Approach	Query Count
Lazy loading (no <code>with()</code>)	1 list query + N relationship queries (N+1)
Eager loading (<code>with('author')</code>)	2 queries total

N+1 vs. Eager Loading: A Visual Comparison

Lazy loading: 16 separate queries



Eager loading: just 2 queries, done



Eager Loading

Eager Loading: loading a relationship up front through `with()` , in a fixed, small number of queries, instead of loading it one row at a time only when it's needed.

```
$articles = Article::with('author')  
    ->latest()  
    ->paginate(15);
```

One extra word, `with('author')` , and those 16 queries immediately shrink to 2. The count also doesn't depend on how many rows get displayed, only on how many relationship levels get loaded, so 15 articles or 150 articles both cost 2 queries total, not 2 multiplied by the row count.

Nested Eager Loading: Dot Notation

A relationship can go more than one level deep. Write it with dot notation, and Eloquent still loads everything in a fixed number of queries:

```
$authors = Author::with('articles.comments')->get();
```

Author



Article



Comment

This one line loads every author, their `articles` relationship, **and** the `comments` relationship inside each article all at once, two levels of relationship in a single dot notation, without the query count growing along with the row count.

Part 4

Pagination for Large-Scale Data

The Basics of Pagination: LIMIT and OFFSET

Displaying 2,500 rows at once on a single page is clearly a bad idea, both for the user and for the database. The solution already exists at the SQL level, well before Laravel gets involved.

LIMIT and OFFSET: SQL clauses that cap the number of rows returned (LIMIT) and skip a number of rows at the start (OFFSET), the basis of every pagination technique, in any framework.

```
-- Halaman 2, 10 baris per halaman  
SELECT * FROM articles  
ORDER BY title  
LIMIT 10 OFFSET 10;
```

rows 1-10 (page 1)

rows 11-20 (page 2)

rows 21-30 (page 3)

OFFSET is computed from the page number via $(\text{page} - 1) * \text{per_page}$: page 1 becomes OFFSET 0, page 2 becomes OFFSET 10, page 3 becomes OFFSET 20, and so on.

Without Laravel: Computing Everything by Hand

Here's what that LIMIT and OFFSET look like written directly in a controller, without `paginate()`'s help:

```
$perPage = 10;
$page = request('page', 1);
$offset = ($page - 1) * $perPage;

$articles = Article::orderBy('title')
    ->skip($offset)
    ->take($perPage)
    ->get();

$total = Article::count();
$lastPage = (int) ceil($total / $perPage);
```

Six lines, and that only computes the paging. Navigation ("page 1 2 3 ...") still has to be written by hand in the view, complete with a link to every page number. This is the exact same work on nearly every listing in any application, so it's no surprise Laravel eventually wrapped it into a single method.

`paginate()` : One Method Replaces All of It

Task	Without Laravel (manual)	With <code>paginate()</code>
Compute offset from page number	Written by hand	Automatic, read from <code>?page=</code>
Count total rows & page count	Separate <code>COUNT()</code> query	Automatic, already included
Page navigation (number links)	Written by hand in the view	<code>{{ \$articles->links() }}</code>

Underneath, `paginate()` still runs the same LIMIT and OFFSET as the previous slide, plus one `COUNT()` query. Laravel just wraps that mechanism so it doesn't have to be rewritten every time.

Applying `paginate()` in a Controller

Those six lines are now a single method call, complete with ready-to-use page metadata:

```
public function index()
{
    $articles = Article::with('author')
        ->orderBy('title')
        ->paginate(10);

    return view('articles.index', compact('articles'));
}
```

```
<!-- resources/views/articles/index.blade.php -->
<div class="mt-4">
    {{ $articles->links() }}
</div>
```

One line, `{{ $articles->links() }}`, in the view is enough, no extra navigation file needs to be built by hand.

When to Use Pagination

Reach for `paginate()` whenever a row count could keep growing without bound over time, like an article listing or a comment history. A plain `get()` is still fine for a listing that's genuinely small and stays small, like a tag dropdown for a filter.

Proving Pagination Works

The code above looks correct, but how do you actually confirm page 2 shows genuinely different rows than page 1, instead of quietly repeating the same data?

```
php artisan tinker
>>> Article::orderBy('title')->paginate(10)->pluck('id');
>>> Article::orderBy('title')->paginate(10, ['*'], 'page', 2)->pluck('id');
```

Compare the results of both commands. Overlapping IDs between page 1 and page 2 signal a problem with the `orderBy` or the data, not with `paginate()` itself.

Applying This to Simple POS

You'll apply the relationship, eager loading, and pagination concepts directly to the Simple POS case study in the practicum jobsheet: mapping the `Category - Product` and `Transaction - TransactionDetail - Product` relationships, avoiding N+1 on a 2,500-transaction listing, and proving pagination across 300 products.

The `Product::take(12)->get()` code you already wrote in Meeting 4 now has a full explanation: it's the basic Eloquent query covered at the start of this meeting.

Full code: github.com/se-polinema/simple-pos-ch05

Summary

- An ORM like Eloquent (using the Active Record pattern) maps a Model class to a table through automatic conventions, with basic CRUD operations and `$fillable` to guard against mass assignment
- Relationships (`hasOne` , `hasMany` , `belongsTo` , `belongsToMany`) are declared once in a model, then used through a method call, property access, or a string inside `with()`
- Lazy loading triggers one extra query per row per relationship (the N+1 problem); eager loading, including nested dot notation, loads a relationship in a fixed query count, independent of row count
- `paginate()` splits a large listing into small pages; different pages can be proven to genuinely differ by comparing result IDs across pages

References & Discussion

Official Laravel documentation (Eloquent, Relationships, Pagination)

Full code: `github.com/se-polinema/simple-pos`

Next meeting: Input Validation & Security